

PRÁCTICA 1

Jerarquía de memoria y comportamiento de memoria caché:

Estudio del efecto de la localidad de los accesos a memoria
en las prestaciones de programas en microprocesadores

ÍNDICE

Introducción	1
Material, Datos y Definiciones	1
Metodología Seguida	3
Resultados Obtenidos	4
Experimentos Adicionales	8
Conclusiones	15

INTRODUCCIÓN

Se nos pide caracterizar el coste temporal por acceso de datos, variando para ello el patrón de acceso a estos y el tamaño del conjunto de valores. De este modo, se estudiará la influencia de las localidades espacial y temporal, además de la precarga, en dichos tiempos de lectura.

Posteriormente, se realizarán experimentos adicionales, variando el tipo de dato de los valores leídos (cambiando, por tanto, el tamaño de su representación binaria) y accediendo de formas distintas a estos.

MATERIAL, DATOS Y DEFINICIONES

Para la realización de los experimentos, se dispone del sistema *Finisterrae III* del CESGA, el cual cuenta con procesadores **Intel Xeon Platinum 8352Y**, con **2.20GHz** de frecuencia. Los datos relativos a dicha CPU han sido obtenidos al ejecutar el comando *lscpu* desde un terminal.

Se consdieran, entonces, los siguientes datos para la toma de resultados:

- **A[]**: vector de datos de tipo *double*, escogidos aleatoriamente en los rangos [1, 2) y (-2, -1].

De este modo, en cada línea caché se tendrá un total de $(64 \text{ bytes} / \text{línea}) / (8 \text{ bytes} / \text{palabra}) = \mathbf{8 \text{ palabras} / \text{línea}}$, al ocupar la representación de *doubles* 8 bytes.

- **S1**: número de líneas en la caché de primer nivel de datos.

Para el procesador empleado, hay un total de **768 líneas** caché L1 de datos, obtenidas de dividir 3 MiB (tamaño de la sección de datos de L1 entre todos los núcleos) entre los 64 bytes que ocupa 1 línea y otra vez entre 64, que es el número de *cores* solicitados.

- **S2**: número de líneas en la caché de segundo nivel.

Para el procesador empleado, se tiene que hay un total de **20480 líneas** caché L2 de datos, obtenidas de dividir 80 MiB (tamaño total de L2 entre todos los *cores*) entre los 64 bytes que ocupa 1 línea, y se divide de nuevo entre 64 (núcleos que hay en total).

- **L**: número de líneas caché diferentes que serán accedidas al leer los valores de $A[]$.

En función de $S1$ y $S2$, se han considerado los siguientes valores de L :

$0.5 * S1$	384
$1.5 * S1$	1152
$0.5 * S2$	10240
$0.75 * S2$	15360
$2 * S2$	40960
$4 * S2$	81920
$8 * S2$	163840

- **R**: número de **elementos del vector $A[]$ que serán accedidos**. Será un múltiplo de $S1$ o de $S2$ en función de si el objetivo es producir muchos o pocos fallos en los niveles $L1$ o $L2$ (en concreto, se desea lograr un número reducido de fallos de primer nivel, elevado en el primer nivel y reducido en el segundo, y, finalmente, elevado en ambos niveles).

Se calculará R como $(8 * L) / D$, en el caso en el que D sea menor que 8, pues en tal caso harían falta $8/D$ saltos para cubrir una línea caché completa (pero queremos acceder a L líneas distintas). Si D ya es mayor o igual que 8, bastará tomar **$R = L$** .

- **D**: potencia de 2 que representa la distancia entre elementos de A , es decir, es un **valor de salto**. Se tomarán valores rondando el 8 para observar cómo varía el acceso a elementos de la misma línea y de las contiguas espacialmente.

Se han escogido los 5 siguientes valores de D :

$D1$	1
$D2$	2
$D3$	4
$D4$	8
$D5$	16

Adicionalmente, se tendrán en cuenta los conceptos de **localidad espacial**, al referenciar un dato es probable que los próximos a este también serán accedidos; y **localidad temporal**, al referenciar un dato es probable que se acceda a este nuevamente en algún instante de la ejecución.

En esta práctica resultará conveniente conocer la técnica de **precarga** o **prefetching**. Esta consiste en un aprovechamiento de los conceptos previamente mencionados de localidad por parte de la caché y la memoria principal, con el fin de intentar **predecir qué datos serán solicitados en el futuro próximo**, de modo que, al pedir un dato, se traerán los cercanos a este al siguiente nivel de la jerarquía de memoria. Esta estrategia se emplea, por ejemplo, trayendo una línea caché y la siguiente en memoria al momento de pedir solo la primera, almacenando la segunda en un *buffer*

auxiliar. De este modo, la precarga resultará útil para esta práctica, al afectar al número de fallos caché que se produzcan, pudiendo alterar los resultados previstos.

En el caso de la CPU considerada para los experimentos, esta pertenece al grupo de **procesadores escalables Intel Xeon de 3ª generación**. Este conjunto presenta un tipo de precarga conocido como **LLC *prefetch*** (información recogida del apartado "*Description*" del siguiente [documento](#)). Dicha técnica consiste en emplear algoritmos que, a partir de patrones previos de acceso a datos, realicen una predicción de las futuras líneas que se pedirán, con lo que se precargarán antes de ser solicitadas ([fuente](#)). Esta estrategia se emplea a nivel de *hardware* con apoyo del *software*, con el fin de **transmitir información entre la memoria principal y la caché de último nivel**. Adicionalmente, existen instrucciones destinadas a la optimización que permiten la realización de *prefetching* en el segundo nivel de la memoria caché (apartados 10.4.6 y 11.6.13 del siguiente [documento](#)).

METODOLOGÍA SEGUIDA

Se detallarán los pasos para obtener los datos pertinentes sobre los accesos a los niveles L1 y L2 de la caché, además de interpretarlos y relacionarlos con los conceptos previos:

- 1) Escribir un código en C para inicializar el vector `A[]` y realizar, sobre este, una **reducción de suma de punto flotante sobre R valores escogidos cada D posiciones**. Se miden los ciclos de reloj requeridos para dichas operaciones y se consideran las medias de resultados y ciclos.
- 2) Escribir un *script* de *bash* que permita la ejecución del programa en C variando los distintos valores de D y R, permitiendo realizar numerosos experimentos a la vez. Dicho *script* compilará el código con la opción de menor optimización, ie, con `-O0`.
- 3) Ejecutar el *script* en un nodo del CESGA, requiriendo para ello reservar memoria suficiente y un número apropiado de núcleos de procesador, que han sido, en este caso, **1 GB** y **64 cores**, respectivamente.

Respecto al tiempo de reserva, se ha probado inicialmente con 1 o media hora, pero debido a sufrir cancelaciones por *timeout* y a largas esperas en la cola de ejecución, se ha optado por pedir **10 o 20 minutos**.

En resumen, se ha accedido a un terminal del CESGA mediante `ssh curso1261@ft3.cesga.es` (o empleando, adicionalmente, un escritorio remoto) y se ha ejecutado el comando `sbatch -n 1 -c 64 -t <tiempo> --mem=1G --job-name p1acg<número> script_experimentos.sh`, para lo que ha sido necesario conceder permisos a los ficheros con los códigos.

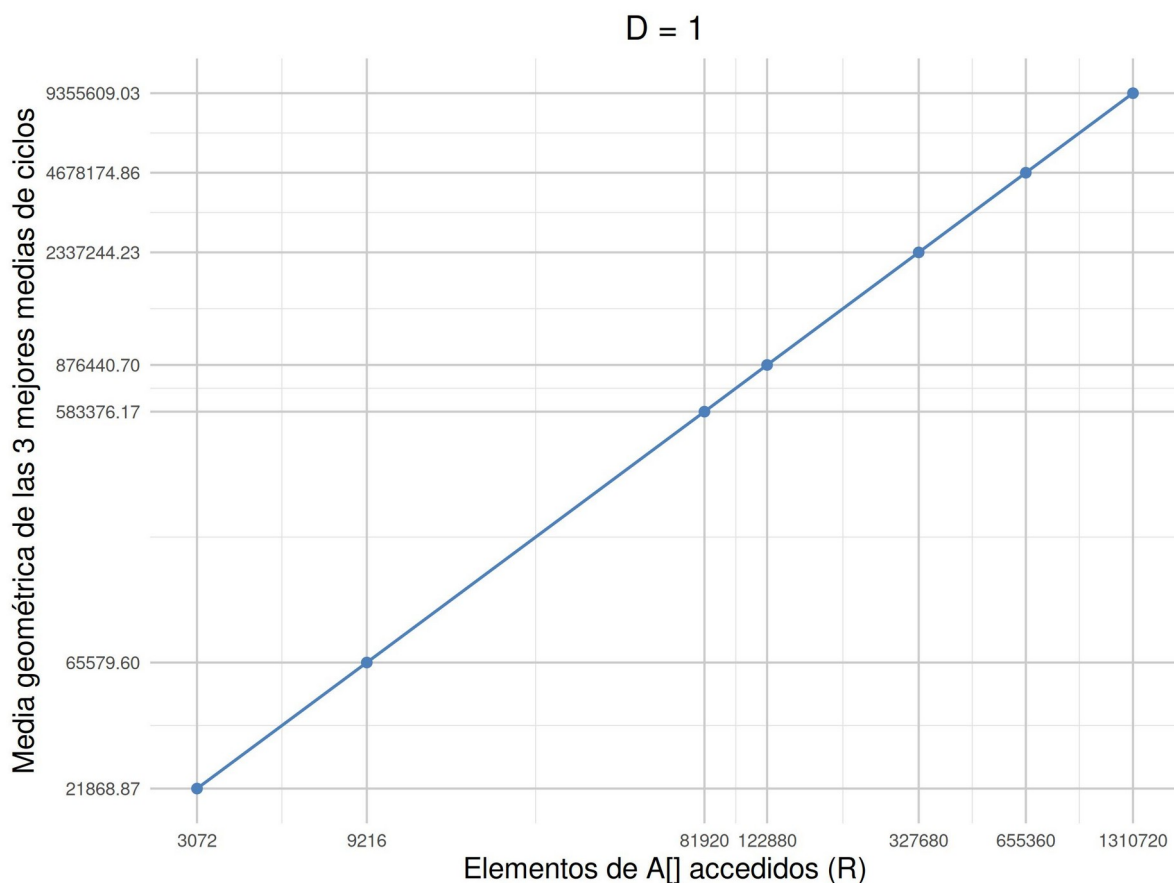
- 4) Rescatar los resultados desde la cuenta pertinente del CESGA y representar los valores obtenidos en gráficas (para lo que se ha empleado R), mostrando la **variabilidad del coste temporal de acceso a los datos en función de las líneas caché referenciadas**.
- 5) Interpretar los resultados, razonando de qué manera han influido los conceptos de localidad en su obtención. También se analiza cómo afecta la precarga a los experimentos.

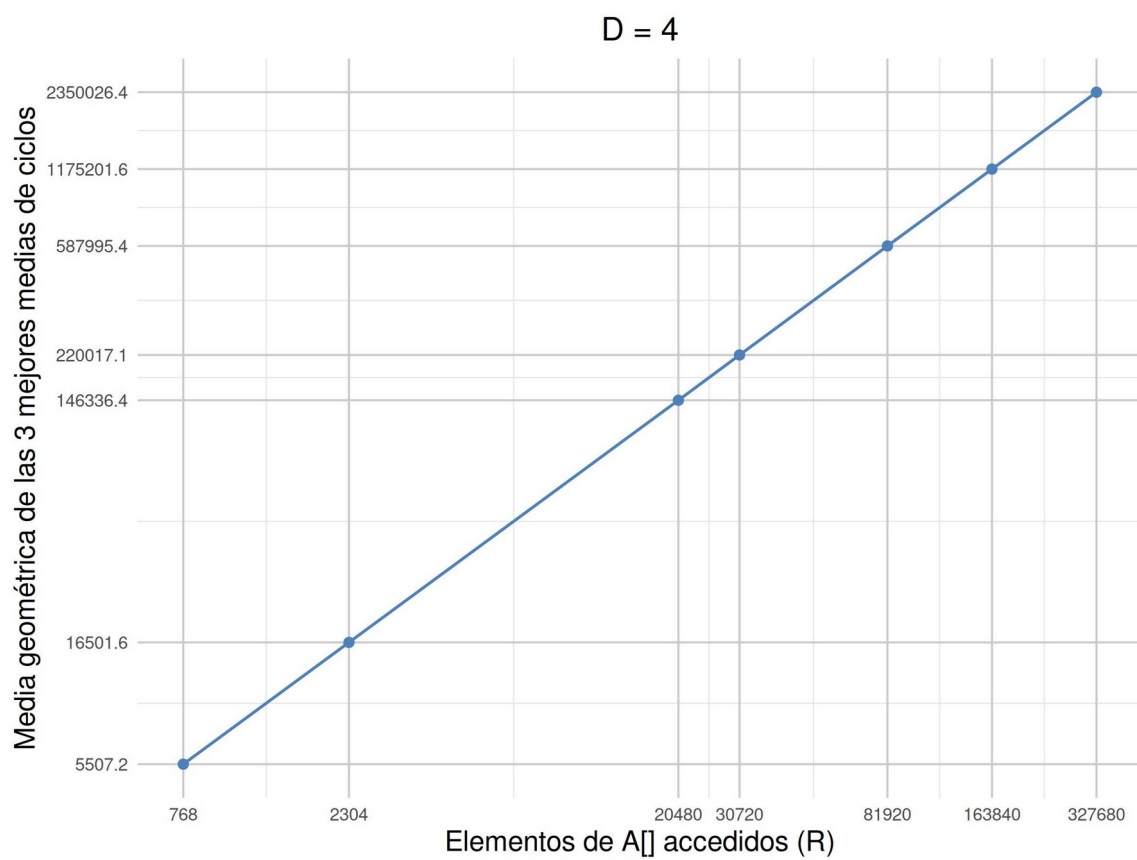
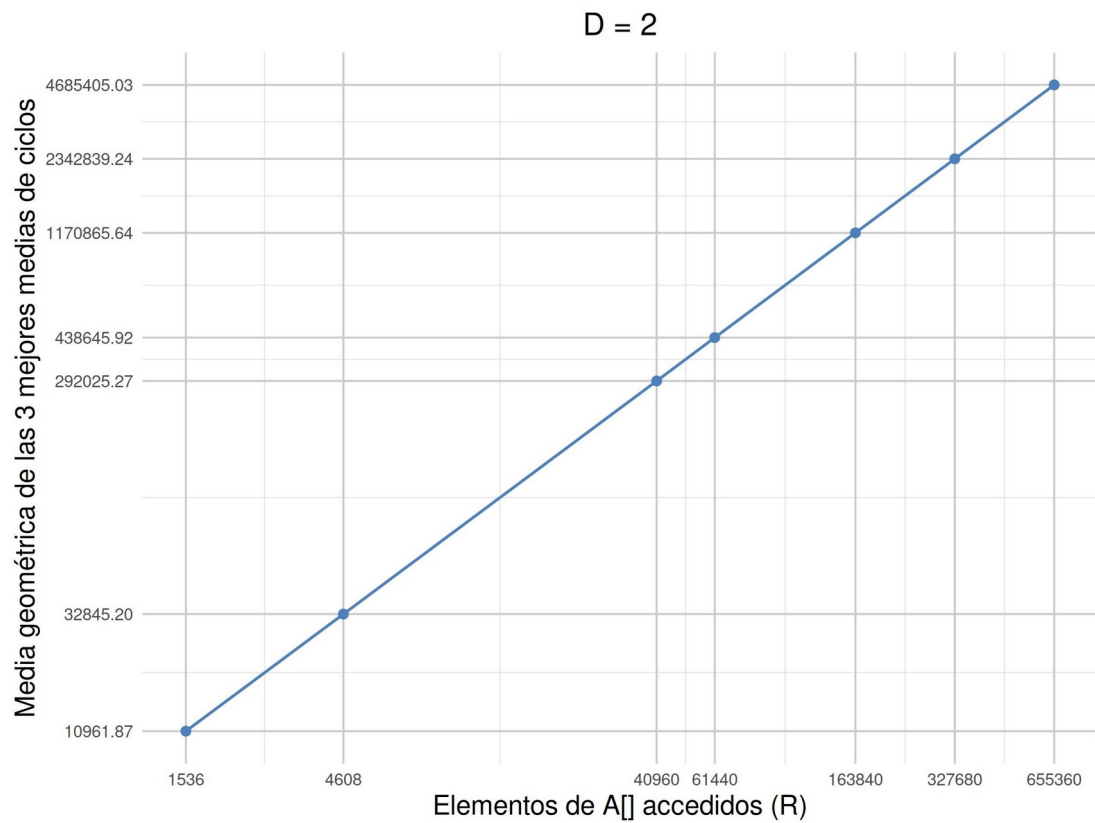
- 6) [Experimentos adicionales] Repetir los pasos previos para códigos donde se accede a las componentes de A[] de manera directa y donde se consideran valores enteros en lugar de flotantes en doble precisión.
- 7) Plasmar todos los datos de interés y las gráficas en este informe.

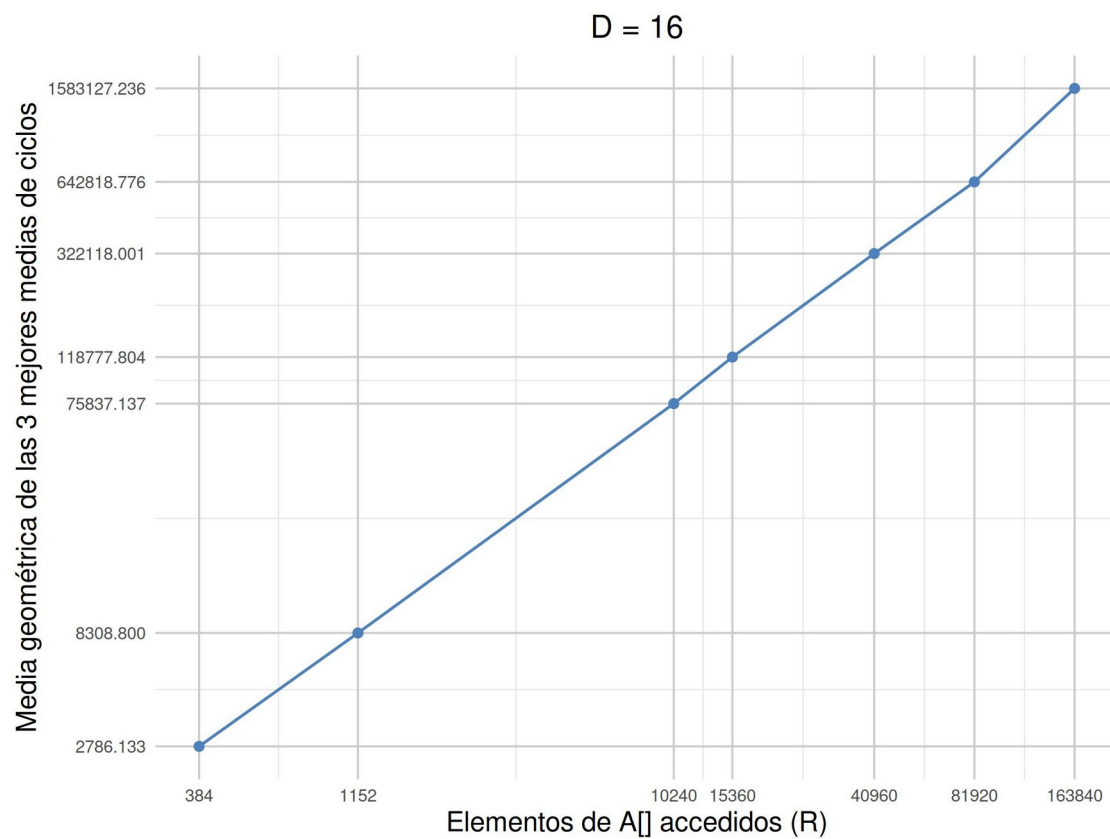
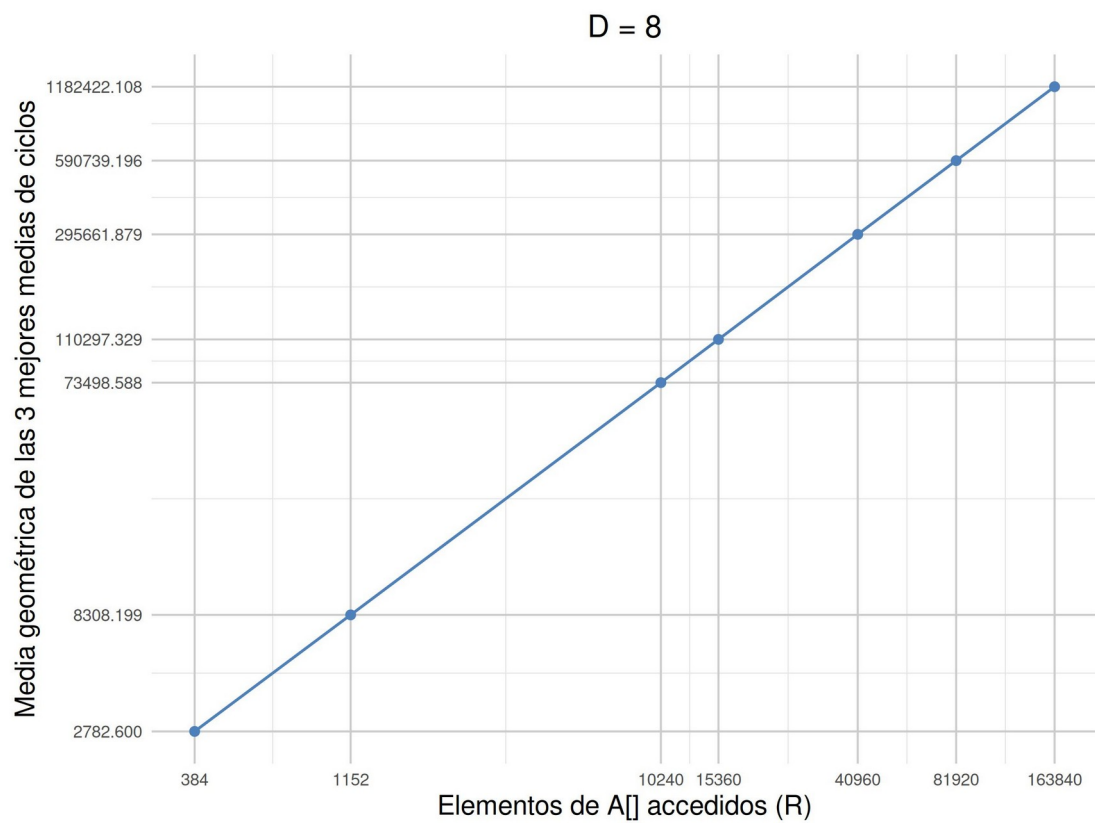
RESULTADOS OBTENIDOS

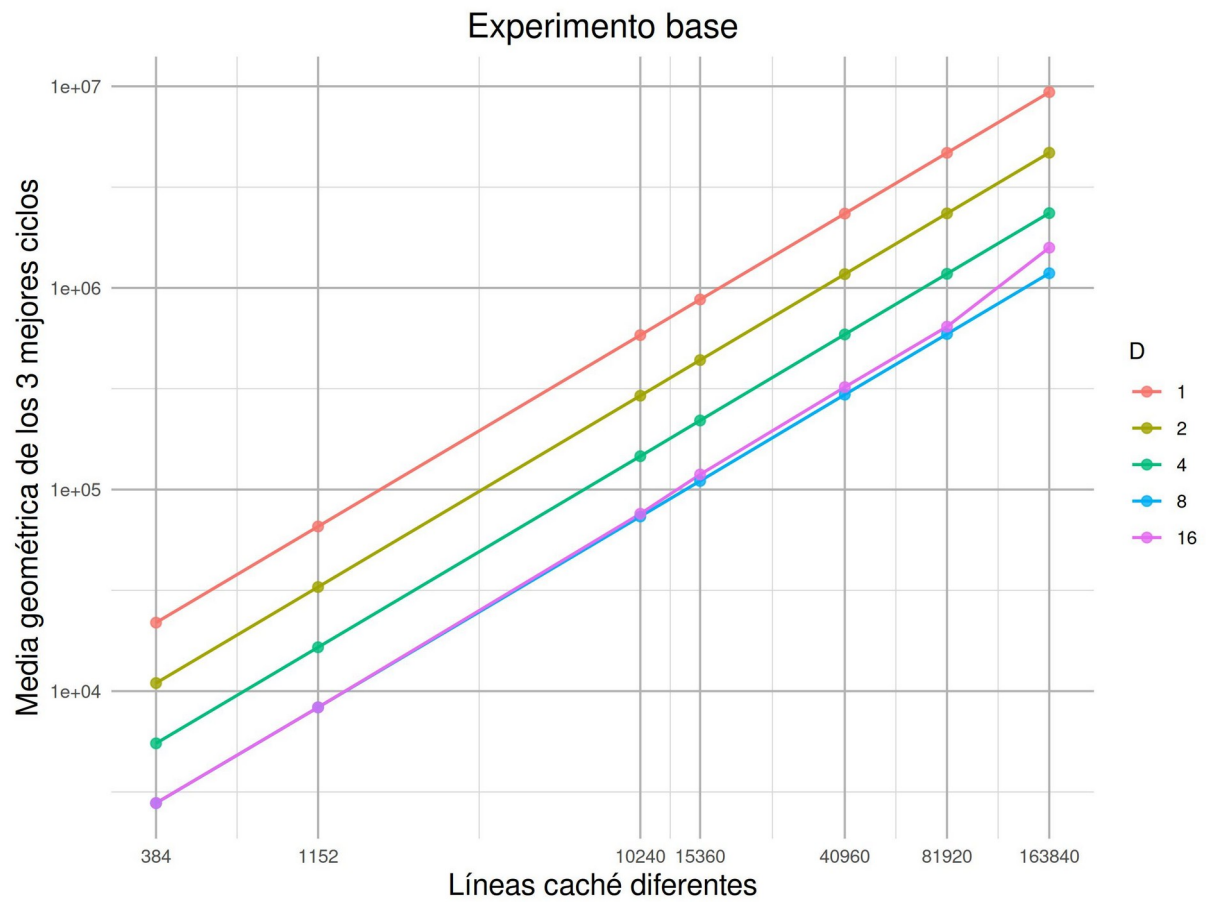
Para el código inicial, se ha obtenido una estimación de la frecuencia de reloj del procesador de **2200.1 MHz $\approx$ 2.20 GHz**, lo que concuerda con la información de la CPU que se ha encontrado a través de la *web*.

En el caso de los ciclos medios (a los que se les ha aplicado una media geométrica entre los 3 valores más bajos), en función de los valores de D y R, se han obtenido los resultados siguientes:





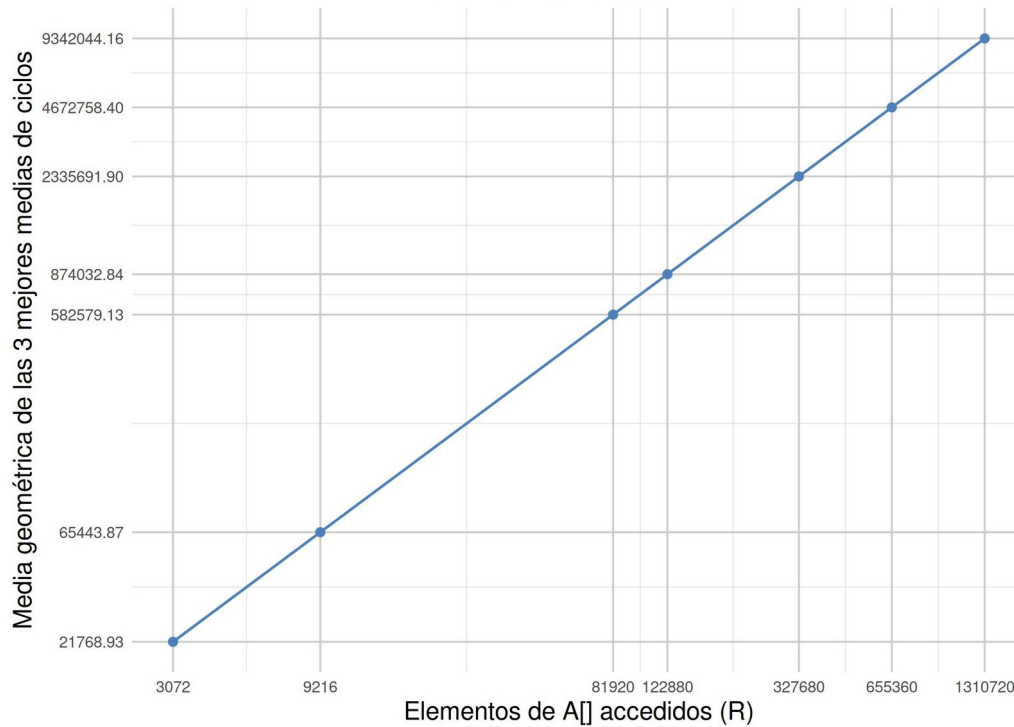




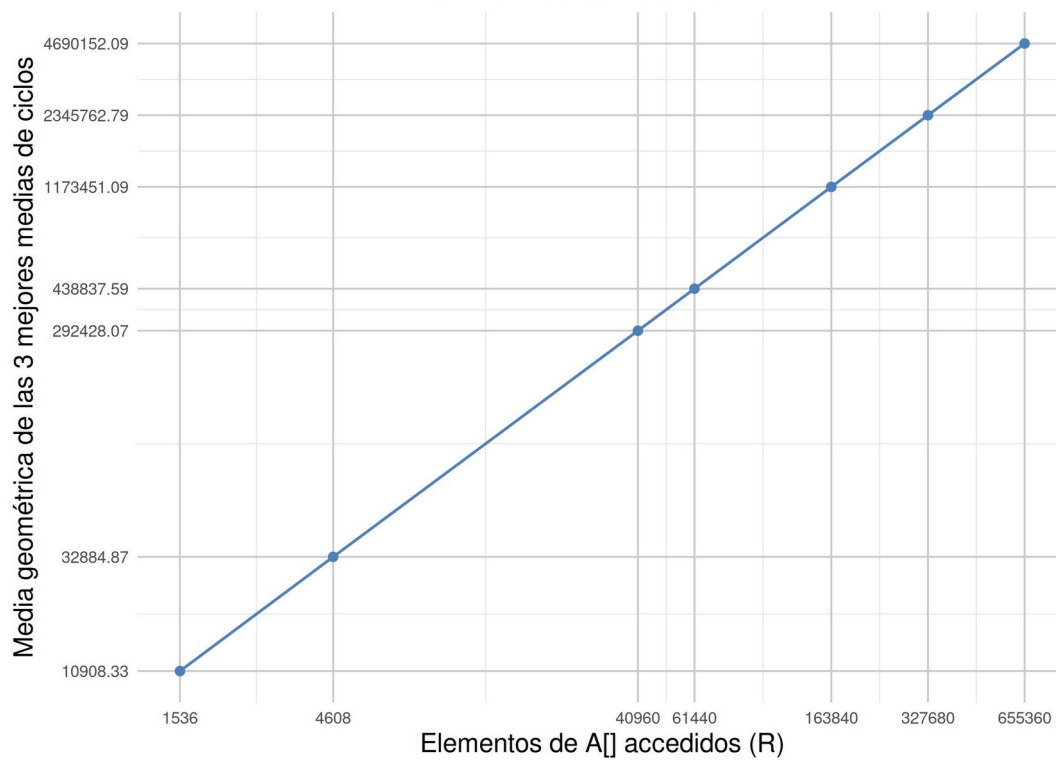
EXPERIMENTOS ADICIONALES

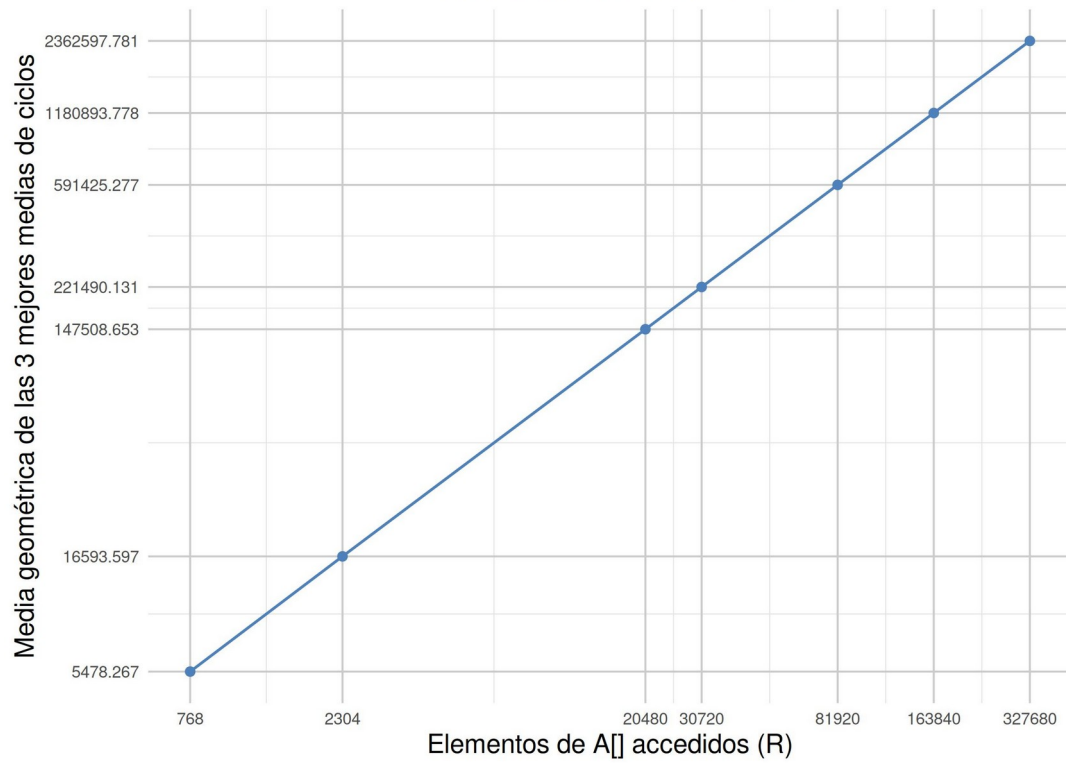
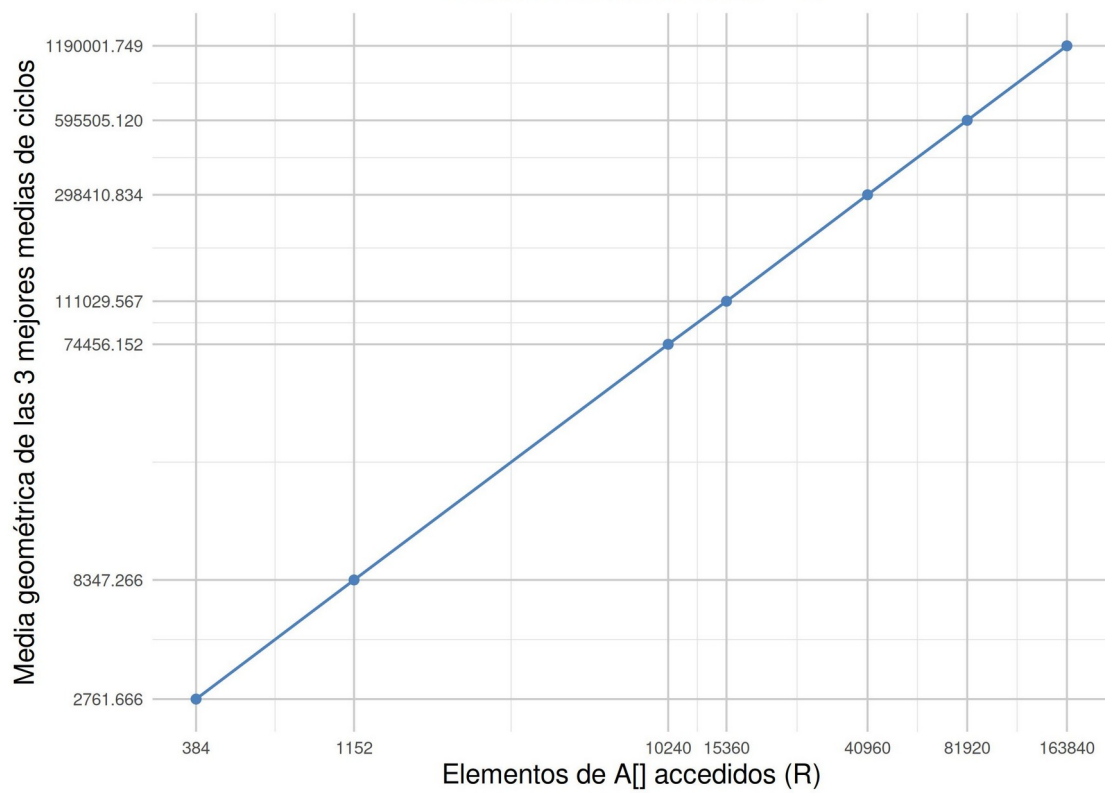
Acceso directo a elementos de A[]:

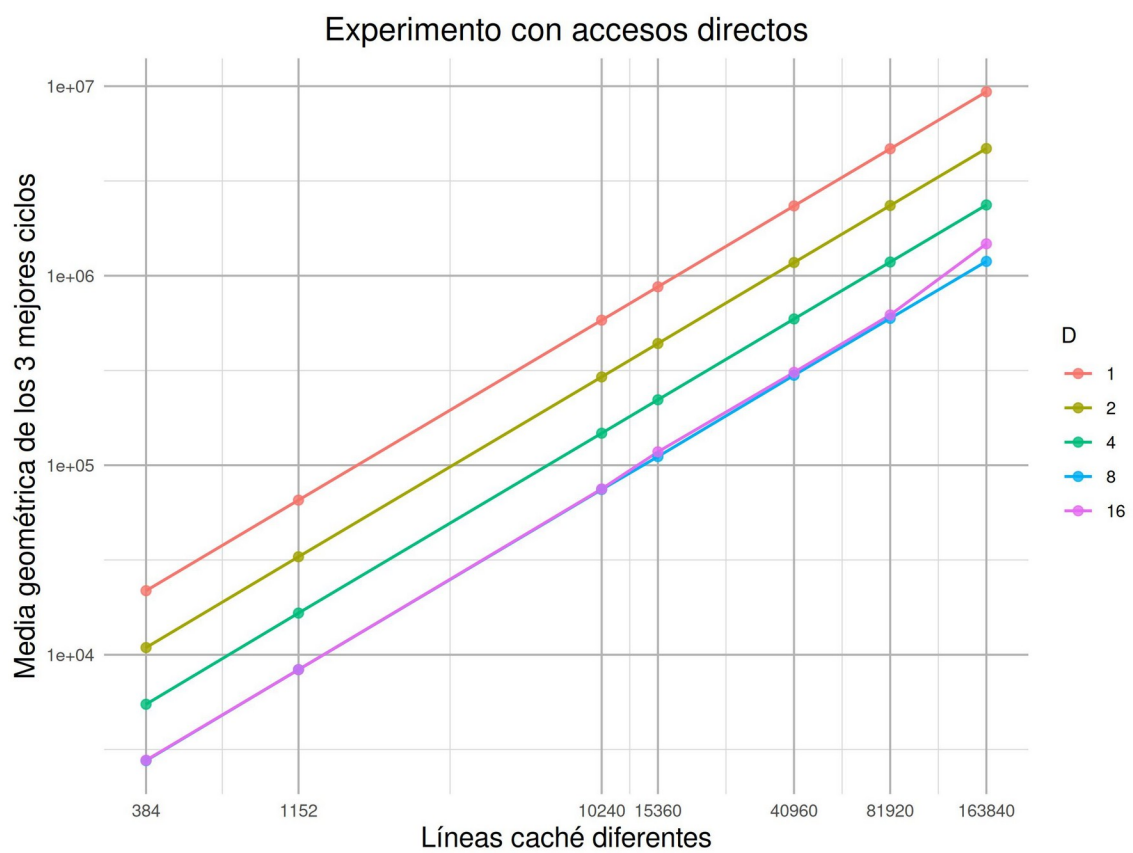
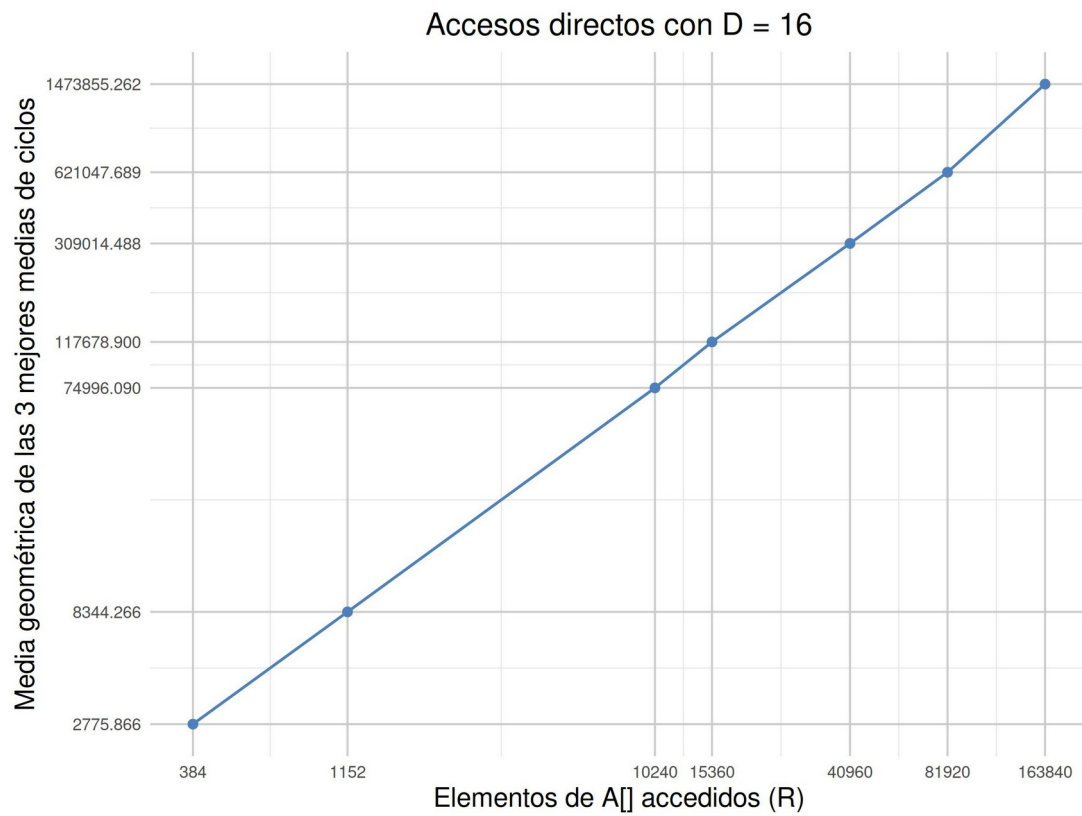
Accesos directos con D = 1



Accesos directos con D = 2



Accesos directos con $D = 4$ Accesos directos con $D = 8$ 

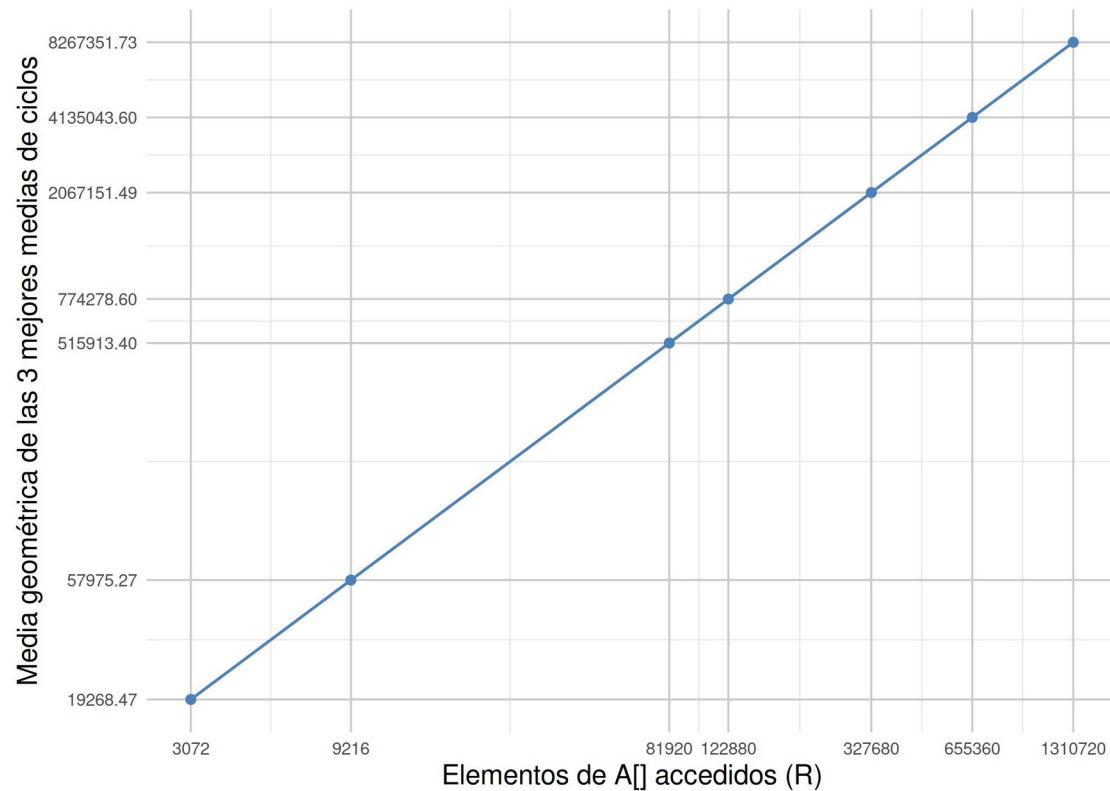


Es apreciable que se mantienen las tendencias del experimento inicial, lo que tiene sentido al solo haber cambiado el modo de acceso a los datos.

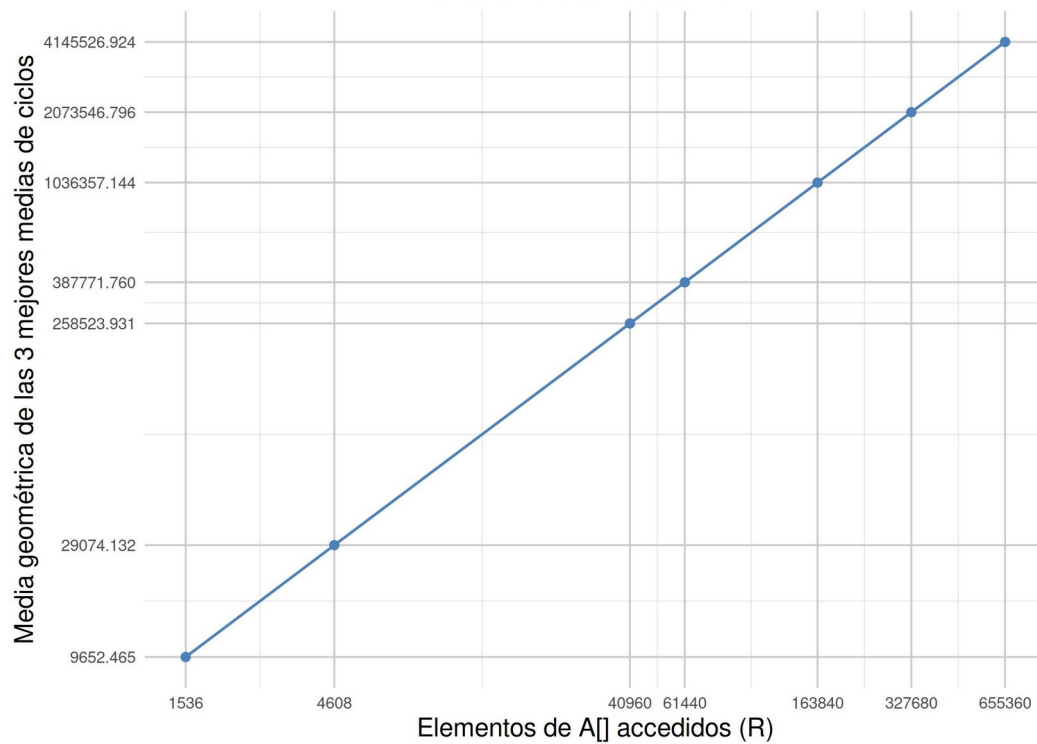
Adicionalmente, se observa que se producen menos fallos de caché al no ser preciso emplear parte de esta para almacenar los elementos del array *ind* (además de que la localidad espacial garantiza que solo es necesario descender en la caché para encontrar datos de *ind* cada cierto tiempo y no continuamente). En promedio, hay una reducción de alrededor del 10% en los ciclos requeridos.

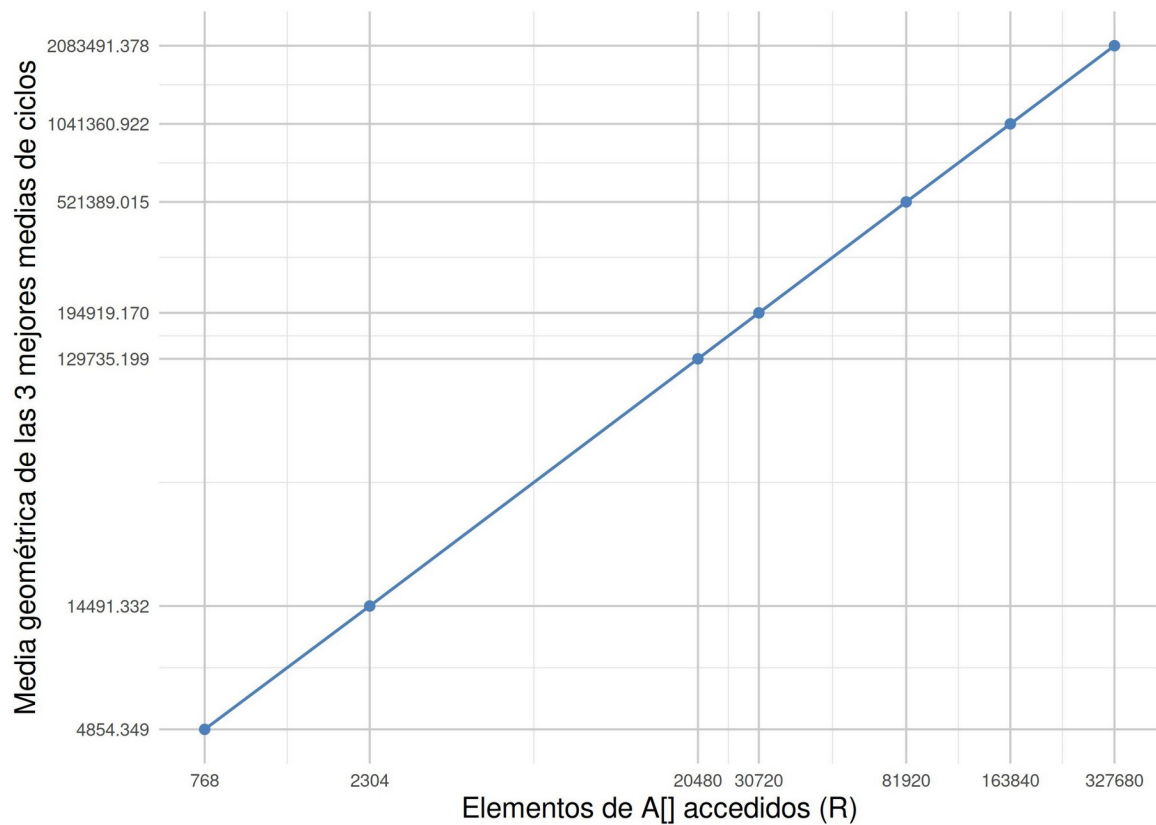
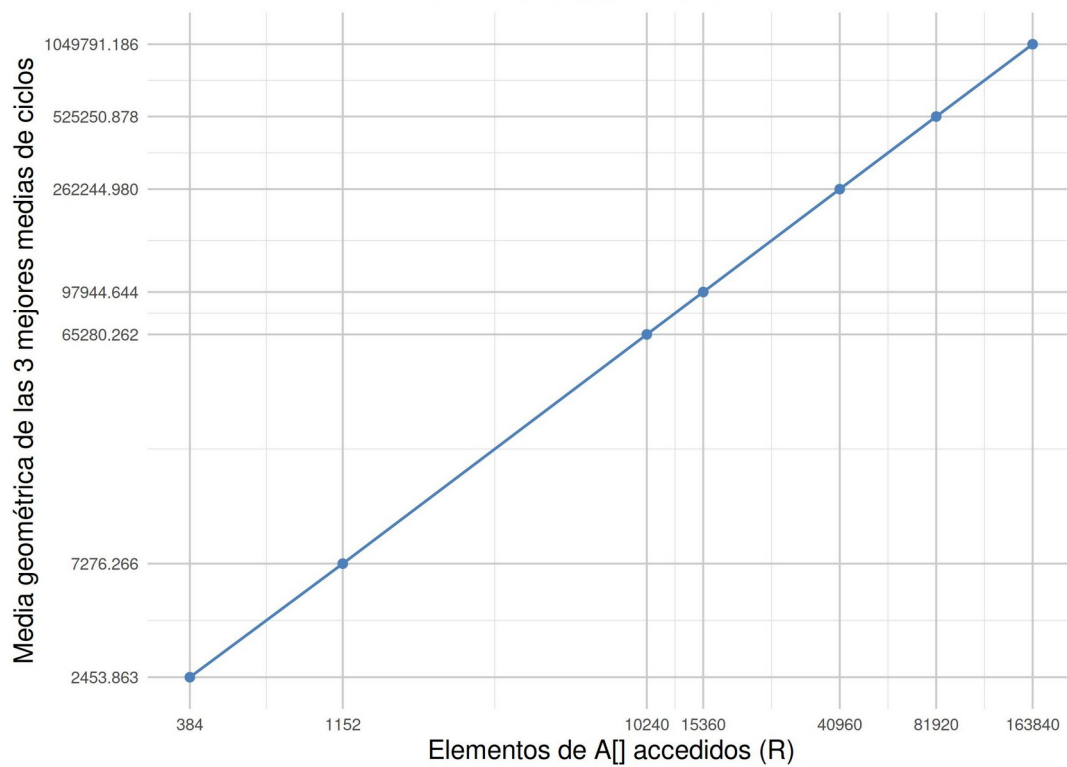
Uso de enteros en lugar de flotantes de doble precisión:

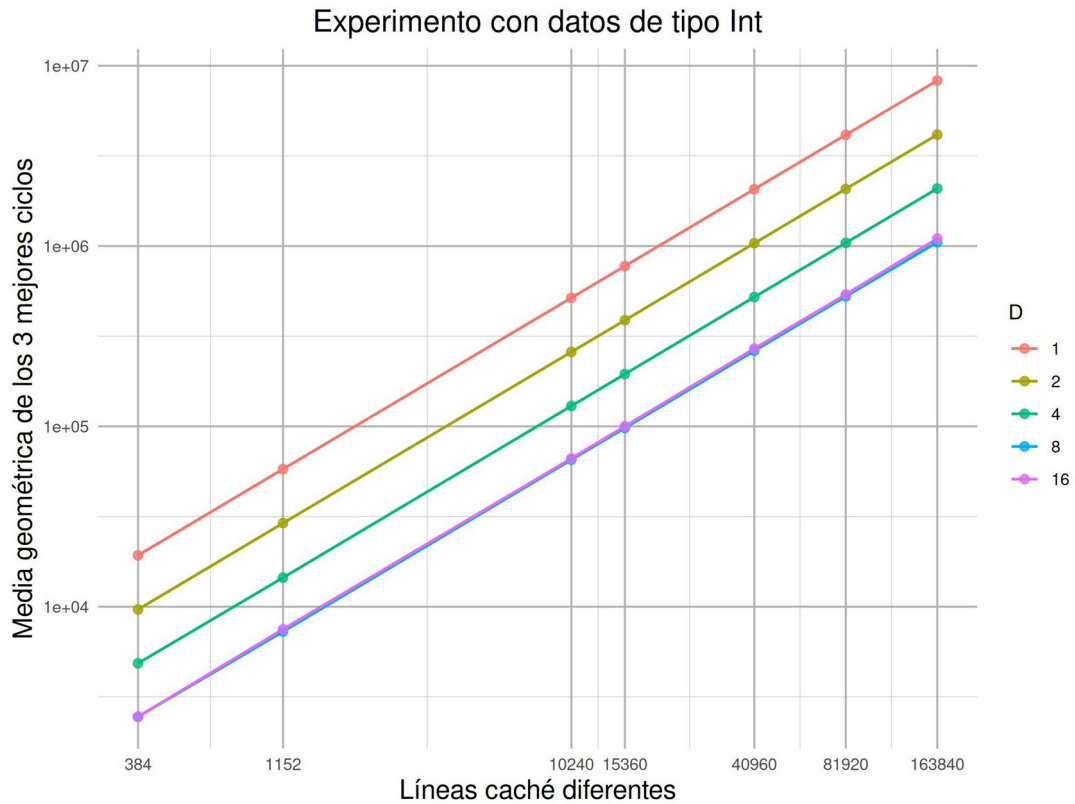
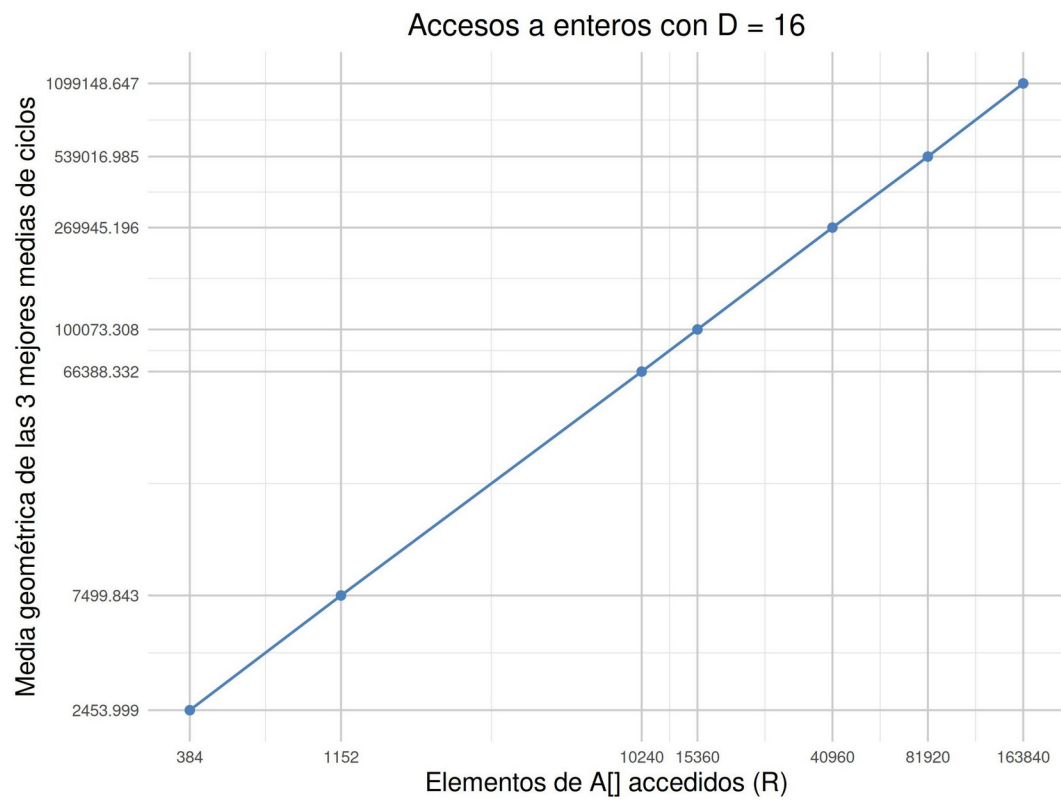
Accesos a enteros con $D = 1$



Accesos a enteros con $D = 2$



Accesos a enteros con $D = 4$ Accesos a enteros con $D = 8$ 



En este caso, se ha calculado R como $(16 \cdot L) / D$, ya que los datos de tipo entero ocupan la mitad que los números en punto flotante de doble precisión, de modo que caben el doble de estos en una línea caché.

Una vez más, se mantiene el crecimiento y las tendencias de los experimentos previos.

El uso de enteros frente a flotantes de doble precisión permite la presencia de una mayor proporción de datos en una misma línea caché, con lo que también será posible presenciar unas medidas de ciclos más reducidas que en el caso base. Es así como se pueden apreciar reducciones de aproximadamente el 20% respecto al experimento de partida.

CONCLUSIONES

Se puede apreciar como, a medida que duplicamos el valor de D , los ciclos medios se reducen aproximadamente a la mitad. Esto es debido a que R es un dato inversamente proporcional a D , lo que obliga a acceder a más datos si se desean cubrir las L líneas caché diferentes. En otras palabras, aunque la localidad espacial esté presente (al tener líneas completas en caché), se accede a los elementos del *array* varias veces por línea, con lo que se suman ciclos de reloj por cada salto (es decir, la contribución temporal viene dada a nivel de registros del procesador, de modo que sumamos más tiempo al tener que acceder a más elementos para cubrir líneas caché diferentes).

Por otra parte, esta tendencia de reducción de ciclos medios desaparece en el momento en el que comenzamos a acceder a datos de líneas distintas, es decir, para D mayores o iguales a 8. Aquí es donde sale a relucir el papel de la precarga, puesto que al traer una línea desde un nivel inferior de la jerarquía de memoria, a este lo acompaña(n) el (o los) siguiente(s), por lo que, para un mayor salto, se siguen accediendo a los mismos niveles caché, lo que deriva en pocas diferencias temporales (por eso los casos $D = 8$ y $D = 16$ no difieren tanto en comparación con los previos).

También resulta conveniente mencionar que, en los primeros experimentos (de los 10 que se hacen para cada par de valores de D y R), es donde se concentra la mayor cantidad de ciclos de reloj para el acceso a datos, puesto que inicialmente se deben traer todos desde niveles inferiores de la jerarquía de memoria, pero, después, la localidad temporal permite que algunas líneas caché se hayan mantenido en niveles superiores, con lo que se ahorran ciclos de reloj.